DownloadIt: Project and Copilot Work Guide

8 October 2026

- DownloadIt: Project and Copilot Work Guide
  - About this guide
  - What can be established about GitHub Copilot's work
  - Project in one sentence
  - Main user journey
  - Architecture at a glance
  - Source-code inventory
  - Key implementation ideas explained
  - API overview
  - What the evidence says is complete, tested, or still limited
  - Recommended way to read the code
  - Companion sections included in the PDF
  - Terms in plain English
- DownloadIt Project Documentation
  - Table of Contents
  - 1. Project Overview
  - 2. Technology Stack
  - 3. Repository Structure
  - 4. Frontend Architecture
  - 5. Backend Architecture
  - 6. API Documentation
  - 7. Authentication
  - 8. Authorization and IDOR Protection
  - 9. Database Design
  - 10. Download Job Lifecycle
  - 11. Provider Architecture
  - 12. yt-dlp Integration
  - 13. Quality Selection
  - 14. FFmpeg and ffprobe
  - 15. File Storage
  - 16. Security Architecture
  - 17. SSRF Protection
  - 18. Error Handling
  - 19. Download Progress
  - 20. History
  - 21. Configuration
  - 22. Docker and Local Infrastructure
  - 23. Local Development Setup
  - 24. Running Tests
  - 25. Verified End-to-End Behavior

  - 26. Test Coverage
  - 27. Frontend UX
  - 28. Current Limitations
  - 29. Production Readiness Assessment
  - 30. Development History
  - 31. Current Project Status
  - 32. Recommended Next Phases

- Architecture Reference
  - Components and Boundaries
  - System Architecture
  - Download Request Lifecycle
  - Provider Selection
  - Job State Machine
  - Deployment Shape

- API Reference
  - Authentication
  - Media Analysis
  - Downloads and History
  - Health and Root
  - Authorization Semantics

- Database Reference
  - Entity Relationship
  - Tables
  - Migration History
  - Lifecycle Semantics

- Security Reference
  - Authentication Flow
  - Ownership and File Authorization
  - SSRF Validation
  - Additional Controls
  - Security Boundaries and Gaps

- Local Development Setup
  - Prerequisites
  - Configure Environment
  - Start Database and Backend
  - Start Frontend
  - Checks and Shutdown

- Testing and Validation
  - Commands
  - Current Audit Results
  - Test Design and Scope
  - Not Established by Tests

- Development History
  - Evidence Available
  - Phase 1–12 Reconstruction
  - Historical Live-Test Evidence
- Phase 13 Report: UI/UX Polish and Real-User Validation
  - 1. Objective
  - 2. Scope guardrails
  - 3. Frontend files updated
  - 4. Implemented UX improvements
  - 5. Implemented connection-error handling
  - 6. Implemented URL validation hardening
  - 7. Implemented auth-page polish
  - 8. Implemented history page responsiveness
  - 9. Implemented result-card state redesign
  - 10. Implemented accessibility updates
  - 11. Documented behavior updates
  - 12. Validation status: frontend lint
  - 13. Validation status: frontend build
  - 14. Validation status: backend tests
  - 15. Live verification performed
  - 16. Connection and infrastructure note
  - 17. What was not live-tested
  - 18. Risk and confidence assessment
  - 19. Final outcome
  - 20. Recommended next phase
- Production Readiness Assessment
  - Operational Risks
  - Suggested Sequence
- Documentation Audit
  - Repository Inspected
  - Documentation Created
  - Current Validation Results
  - Verified Implemented Features
  - Verified Security Controls
  - Verified Download Providers
  - Verified Live-Test Evidence
  - Incomplete / Planned Features
  - Configuration Warnings
  - Production-Readiness Warnings
  - Documentation Confidence

DownloadIt: Project and Copilot Work Guide

Detailed project handover and source-code map
Prepared from the project files and documentation available in this workspace

Date: 8 October 2026

About this guide

This guide explains the DownloadIt application, how its main code areas fit together, what the existing project records say about development work, and how to run and validate it. The appendix maps each application, test, and migration source file to its responsibility. The companion reference sections provide the API contract, architecture, data model, security controls, setup, test evidence, limitations, and production-readiness notes.

This is a system-level code explanation, not a verbatim commentary on every source line. For exact implementation details, follow the file links in the source tree and the descriptions in the module inventory below.

What can be established about GitHub Copilot's work

The checked-out workspace does not contain Git metadata. Therefore, it does not provide commits, authors, pull requests, or change attribution. The files also do not independently prove whether a particular change was authored by GitHub Copilot, a person, or both. This guide does not assign authorship without evidence.

The strongest project-specific work record available is PHASE_13_REPORT.md. It documents a frontend UX and accessibility pass, including clearer API connection errors, URL-scheme validation, mobile navigation/history improvements, result-state redesign, reduced-motion/focus treatments, and updates to user-facing claims. It also records validation claimed for that phase: frontend lint/build, 92 backend tests, and a local end-to-end Direct MP4 sample flow. The report explicitly says that YouTube and Instagram were not live-tested and that the direct sample does not establish broad platform support.

DEVELOPMENT_HISTORY.md states that exact numbered implementation history for earlier phases could not be reconstructed. The three Alembic migration numbers are database schema revisions, not evidence of a complete product timeline. Consequently, the reliable answer to "what Copilot did" is: the repository documents what the project contains and what Phase 13 reports as completed, but it cannot verify Copilot authorship or a complete Copilot activity log.

Project in one sentence

DownloadIt is a React and TypeScript web app with a FastAPI service that analyzes media URLs, lets a signed-in user create a temporary download job for an offered format, tracks progress, and serves completed files only through an authenticated owner-checked endpoint.

Main user journey

The browser opens the landing page and submits a URL to `POST /api/media/analyze`.
The backend chooses an enabled provider. The direct-file provider is restricted to exact HTTPS host allowlisting; the optional yt-dlp provider is disabled by the example configuration.
The browser renders only formats returned by analysis. It does not send raw yt-dlp selector expressions.
A signed-in user creates a job. For yt-dlp, the request also requires an authorization

acknowledgement.
FastAPI inserts a `pending` job row and starts an in-process background task.
The worker downloads into private temporary storage, reports progress, checks size and output paths, and uses ffprobe to validate the actual container and stream.
The browser polls the owner-scoped job status and retrieves a completed, unexpired file through the authenticated file endpoint.
History lists the signed-in user's jobs. Media bytes are not stored in PostgreSQL.

Architecture at a glance

```
React browser
   ├── pages, forms, analysis and progress UI
   ├── API client and sessionStorage bearer token
   └── FastAPI routes
         ├── authentication and owner checks
         ├── services and PostgreSQL metadata
         ├── provider registry
         │     ├── allowlisted direct MP4
         │     └── optional yt-dlp integration
         ├── in-process background worker
         ├── ffprobe output validation
         └── private temporary files
```

The app uses React state and a small authentication context rather than a global state/query library. The frontend selects views from `window.location.pathname`; no routing package is configured. On the server, routes validate inputs and delegate database or provider work to services. PostgreSQL holds accounts and job metadata, while local disk holds temporary media.

Source-code inventory

Frontend entry, routing, and shared state

File

Responsibility

`src/main.tsx`

Browser entry point; mounts the app under React Strict Mode and imports global styling.

`src/App.tsx`

Chooses the page from the pathname, provides the auth context, and gates History while auth state is loading or signed out.

`src/context/auth-context.ts`

Defines the shared authentication context contract.

`src/context/auth-provider.tsx`

Owns signed-in user/loading state, restores a saved token through `/api/auth/me`, and exposes login/logout behavior to child components.

`src/context/useAuth.ts`

Typed hook for consuming the auth context and detecting use outside its provider.

`src/lib/authStorage.ts`

Gets, saves, and clears the short-lived browser-session bearer token in `sessionStorage`.

`src/lib/api.ts`

Central API client: base URL resolution, network/error handling, JSON requests, authentication headers, auth calls, analysis, job creation/status, history, and file retrieval.

`src/types/media.ts`

TypeScript request/response contracts for analysis and offered media formats.

`src/types/download.ts`

TypeScript contracts for a created job, status/progress, and history items.

Frontend pages and components

File

Responsibility

`src/pages/Home.tsx`

Assembles the landing page and URL download flow.

`src/pages/Login.tsx`

Login form, field handling, validation messages, and auth-provider integration.

`src/pages/Register.tsx`

Registration form and password-policy guidance, submitting through the API client.

`src/pages/History.tsx`

Loads the current user's history, displays statuses/details responsively, and offers file retrieval for completed jobs.

`src/components/UrlDownloader.tsx`

Controls URL analysis, the selected offered format, acknowledgement, job submission, polling, and progress/error/success states.

`src/components/MediaResult.tsx`

Displays analysis metadata, available formats, job states, progress, and completion affordances.

`src/components/Navbar.tsx`

Responsive top navigation and sign-in/history navigation state.

`src/components/Hero.tsx`

Landing-page introduction and primary call to action.

`src/components/Features.tsx`

Renders the landing-page feature section from local feature data.

`src/components/FeatureCard.tsx`

Reusable feature card that accepts an icon, title, and explanatory text.

`src/components/Logo.tsx`

Shared brand mark/wordmark component.

`src/components/Footer.tsx`

Shared page footer and footer navigation.

`src/index.css`

Global Tailwind CSS import and base styles.

`src/App.css`

App-level styles, where applicable.

`src/assets/hero.png`

Hero artwork used by the frontend.

`src/assets/react.svg`, `src/assets/vite.svg`

Starter/template assets; their presence does not imply React/Vite marketing content is part of the product flow.

Backend application, APIs, and contracts

File

Responsibility

`backend/app/main.py`

Creates FastAPI, configures CORS, includes routers, maps database errors to a safe unavailable response, and exposes the root service response.

`backend/app/api/dependencies.py`

Resolves and validates the bearer-token user dependency used by protected routes.

`backend/app/api/routes/auth.py`

Registration, login, and current-user API endpoints.

`backend/app/api/routes/media.py`

Public media-analysis endpoint; validates the request and delegates to provider selection.

`backend/app/api/routes/downloads.py`

Owner-scoped history, job creation, details, status, file download, and deletion endpoints.

`backend/app/api/routes/health.py`

Health endpoint, including database availability.

`backend/app/api/routes/__init__.py`, `backend/app/api/__init__.py`, `backend/app/__init__.py`

Python package markers for the API and application packages.

`backend/app/schemas/auth.py`

Pydantic registration/login/user/token schemas and email normalization/password validation.

`backend/app/schemas/media.py`

Pydantic analysis request/response and format-option schemas.

`backend/app/schemas/downloads.py`

Pydantic job create, status, history, detail, and delete response schemas.

`backend/app/schemas/__init__.py`

Schema package marker.

`backend/app/models/user.py`

SQLAlchemy user model, normalized email uniqueness, password hash, active state, and relationship to downloads.

`backend/app/models/download.py`

SQLAlchemy job/history model, ownership, state, format, timestamps, progress counters, and relative file-path metadata.

`backend/app/models/__init__.py`

Model package marker/exports.

Backend configuration, providers, services, and workers

File

Responsibility

`backend/app/core/config.py`

Typed application settings loaded from environment, including database, JWT, provider, storage, and size/expiry options.

`backend/app/core/database.py`

SQLAlchemy base, engine/session setup, and request-scoped database session dependency.

`backend/app/core/network_security.py`

Connection-time DNS resolution guard that rejects non-public addresses during guarded provider work.

`backend/app/core/__init__.py`

Core package marker.

`backend/app/providers/base.py`

Shared provider protocol and data structures for platform identity, media formats, and metadata.

`backend/app/providers/registry.py`

Ordered provider lookup; chooses the first enabled provider matching the requested URL.

`backend/app/providers/direct_file.py`

Direct HTTPS MP4 provider constrained by exact host allowlist, MIME checks, no redirects, streaming, and size limits.

`backend/app/providers/ytdlp_provider.py`

Optional yt-dlp analysis/download adapter, controlled format mapping, public URL checks, playlist rejection, authorization requirement, progress, and safe error conversion.

`backend/app/providers/errors.py`

Typed provider failure categories and mapping from safe provider errors to HTTP responses.

`backend/app/providers/__init__.py`

Provider package marker/exports.

`backend/app/services/auth_service.py`

Password hashing/verification, user creation and lookup, authentication, and JWT issue/validation.

`backend/app/services/media_service.py`

Provider selection and analysis orchestration for a URL.

`backend/app/services/download_service.py`

Job creation/owner lookup, state transitions, progress persistence, and lazy expiration.

`backend/app/services/job_queue.py`

Queue interface and current in-process background-task adapter.

`backend/app/services/storage_service.py`

Private local temporary storage, generated names, file permissions, path validation, and containment.

`backend/app/services/storage_cleanup.py`

Callable cleanup of expired files and orphaned storage; the project documentation says it is not scheduled automatically.

`backend/app/services/__init__.py`

Service package marker.

`backend/app/workers/download_worker.py`

Processes jobs, updates state/progress, downloads through the provider, enforces output checks, runs ffprobe validation, and records safe completion/failure.

`backend/app/workers/__init__.py`

Worker package marker.

Database migration and backend test files

File

Responsibility

`backend/alembic/env.py`

Connects Alembic to application settings and model metadata for migration operations.

`backend/alembic/script.py.mako`

Template Alembic uses when generating a revision.

`backend/alembic/versions/0001_initial_download_history.py`

Creates initial users and downloads tables and their base ownership/history structure.

`backend/alembic/versions/0002_user_authentication.py`

Adds authentication fields, normalizes existing email data, and enforces case-insensitive uniqueness.

`backend/alembic/versions/0003_download_progress.py`

Adds downloaded/total byte counters and percentage progress.

`backend/tests/conftest.py`

Shared isolated test database, FastAPI dependency overrides, fixtures, and test defaults.

`backend/tests/test_auth.py`

Registration/login/password/token behavior and user-scoped authorization tests.

`backend/tests/test_downloads.py`

Download API creation, history, status, ownership, and failure/validation behaviors.

`backend/tests/test_download_worker.py`

Worker transitions, progress, file storage/validation, completion/failure, expiry, and cleanup behavior.

`backend/tests/test_health.py`

Health endpoint and unavailable-database response behavior.

`backend/tests/test_providers.py`

Registry and direct-provider URL, response, MIME, redirect, and network behavior.

`backend/tests/test_ytdlp_provider.py`

yt-dlp provider behavior using fakes for metadata, options, URLs, quality, limits, and errors.

`backend/tests/__init__.py`

Test package marker.

Project-level files and infrastructure

File or folder

Responsibility

`package.json` / `package-lock.json`

Frontend dependencies and `dev`, `build`, `lint`, and `preview` scripts with locked versions.

`vite.config.ts`

Vite frontend build/dev-server configuration.

`tsconfig.json`, `tsconfig.app.json`, `tsconfig.node.json`

TypeScript project and frontend/tooling compiler settings.

`index.html`

Vite HTML entry document.

`docker-compose.yml`

Starts PostgreSQL only; frontend and API are separate local processes.

`backend/alembic.ini`

Alembic configuration.

`backend/requirements.txt`

Pinned base Python backend dependencies.

`backend/requirements-ytdlp.txt`

Separate optional yt-dlp dependency.

`.env.example`, `backend/.env.example`, `backend/.env.docker`

Configuration examples/separation; actual ignored local secret files are intentionally excluded from this guide.

`public/favicon.svg`, `public/icons.svg`

Public browser icon assets.

`README.md`, `backend/README.md`

Local project/backend orientation; read alongside the more detailed docs for current route behavior and caveats.

Key implementation ideas explained

Providers keep source-specific behavior behind a common contract

`MediaProvider` describes shared operations. `ProviderRegistry` uses the configured order to select one enabled provider. This avoids putting media-source protocol details in route handlers. The direct provider is intentionally narrow. yt-dlp is an opt-in general extractor adapter, not a promise of universal site support.

Routes, schemas, and services have different jobs

Pydantic schemas define accepted/returned API shapes. Route functions bind HTTP methods and paths to those contracts, enforce authentication where required, and translate expected errors. Services own reusable business operations such as job state changes or storage. The worker executes long-running work outside the request function, though its current queue is still in-process rather than durable.

Database rows describe jobs; local storage holds media

The `users` table represents accounts. Each `downloads` row belongs to one user and records the source, selected output, status, progress, timestamps, expiry, and a relative file path. Three migrations evolve this schema. The media itself is not stored in PostgreSQL.

Output is validated before success is reported

A successful provider return alone is not enough. The worker checks that the generated file exists, is not a symlink or empty, stays under the size limit, has no unexpected residual files, and passes ffprobe checks for a valid MP4/M4A container and required stream. Failed jobs expose safe messages rather than raw provider diagnostics.

Security is layered, with deployment caveats

Protected job and file operations scope database lookups by both job ID and authenticated user ID. Storage uses generated identifiers and constrained paths. URL access is guarded against private-address SSRF; the direct provider is further restricted to configured exact hosts and disallows redirects. The codebase also documents important limits: browser tokens are in JavaScript-readable `sessionStorage`, direct HTTP requests may honor proxy environment variables, rate limiting is absent, and local storage/in-process work are not production-scale infrastructure.

API overview

Method

Path

Purpose

`POST`

`/api/auth/register`

Create an account.

`POST`

`/api/auth/login`

Authenticate and receive a bearer token.

`GET`

`/api/auth/me`

Validate the current token and return its active user.

`POST`

`/api/media/analyze`

Analyze a URL and return provider metadata/formats; this route is public in the current source.

`GET`

`/api/downloads`

List the signed-in user’s job history.

`POST`

`/api/downloads`

Create an owner-bound download job.

`GET`

`/api/downloads/{id}`

Read owner-scoped job details.

`GET`

`/api/downloads/{id}/status`

Read job state and progress.

`GET`

`/api/downloads/{id}/file`

Retrieve an owned, complete, unexpired file.

`DELETE`

`/api/downloads/{id}`

Delete an eligible owned job.

`GET`

`/api/health`

Check API/database health.

`GET`

`/`

Return service name/version.

The complete field definitions and status/error semantics are in API.md.

What the evidence says is complete, tested, or still limited

- The project documentation reports a React frontend, FastAPI API, PostgreSQL metadata schema, authentication, provider registry, job worker, local temporary storage, and file validation.
- Three Alembic revisions are present; `0003_download_progress` is described as the current migration head.
- The documentation records 92 backend tests plus frontend lint/build results for the prior validation run. Those are historical recorded results, not a claim that this guide re-ran them.
- The backend tests use fake providers, mocked HTTP/DNS, an isolated test database, and mocked ffprobe where relevant. They verify application behavior but cannot guarantee a live

website extractor still works.

The Phase 13 report records one local Direct MP4 sample flow. It explicitly does not claim YouTube or Instagram live verification.

The worker queue is process-local. Cleanup exists but has no automatic scheduler. No production deployment, rate limiter, observability stack, database backup plan, or frontend browser-test suite is documented.

yt-dlp is optional and disabled in the example configuration. Use it only for media you are authorized to download and where the source permits it. No DRM, access-control, login, CAPTCHA, or rate-limit bypass is implemented.

Recommended way to read the code

Begin at `src/main.tsx` and `src/App.tsx` to understand the browser entry and page selection.

Follow `src/components/UrlDownloader.tsx` to the API methods in `src/lib/api.ts` and the types in `src/types/`.

Read `backend/app/main.py`, then the route modules and Pydantic schemas.

Trace job creation through `download_service.py`, `job_queue.py`, and `workers/download_worker.py`.

Read `providers/base.py`, `registry.py`, and provider implementations to understand analysis/download dispatch.

Read `models/`, `alembic/versions/`, and `services/storage_service.py` to understand persisted metadata and temporary media.

Use the companion API, security, database, testing, setup, and architecture references for precise contracts and boundaries.

Companion sections included in the PDF

The full PDF continues with the project's existing detailed references:

PROJECT_DOCUMENTATION.md — complete system overview and feature-by-feature explanation.

ARCHITECTURE.md — component boundaries, request sequence, provider selection, job state, and deployment shape.

API.md — endpoint contracts, request/response fields, and error behavior.

DATABASE.md — tables, columns, relationships, migration sequence, and lifecycle semantics.

SECURITY.md — authentication, ownership, SSRF, storage, and security gaps.

SETUP.md — local environment and run instructions.

TESTING.md — test design, recorded results, and what tests do not establish.

DEVELOPMENT_HISTORY.md — available evidence and limits on historical attribution.

PHASE_13_REPORT.md — detailed record of the Phase 13 UI/UX work and its reported validation.

PRODUCTION_READINESS.md — implementation gaps and operational recommendations.

DOCUMENTATION_AUDIT.md — documented inspection scope, verified behavior, and confidence caveats.

Terms in plain English

Term	Meaning in this project
API	HTTP interface the browser calls to analyze URLs, authenticate, and manage jobs.
Bearer token/JWT	Signed short-lived credential attached to protected API requests.
Provider	Component that knows how to inspect/download one category of media source.
Registry	Chooses an enabled provider for a URL.
Job	Database record representing one requested download and its processing status.
Worker	Function that performs provider download and final media checks.
ffmpeg	External media tool yt-dlp can use to merge compatible streams.
ffprobe	External media inspector used by the worker to validate completed output.
SSRF	Server-side request forgery: tricking a server into connecting to unintended network addresses.
IDOR	Insecure direct object reference: accessing another user’s record by guessing/changing an ID.
Alembic migration	

Versioned database schema change that can be applied in order.

In-process background task

Work run by the API process itself; simple locally, but not a durable distributed queue.

DownloadIt Project Documentation

Table of Contents

Project Overview
Technology Stack
Repository Structure
Frontend Architecture
Backend Architecture
API Documentation
Authentication
Authorization and IDOR Protection
Database Design
Download Job Lifecycle
Provider Architecture
yt-dlp Integration
Quality Selection
FFmpeg and ffprobe
File Storage
Security Architecture
SSRF Protection
Error Handling
Download Progress
History
Configuration
Docker and Local Infrastructure
Local Development Setup
Running Tests
Verified End-to-End Behavior
Test Coverage
Frontend UX
Current Limitations
Production Readiness Assessment
Development History
Current Project Status
Recommended Next Phases

1. Project Overview

DownloadIt is a React/TypeScript web client and FastAPI service for analyzing media URLs and preparing temporary, authenticated downloads. It stores account and job metadata in PostgreSQL; media files are stored locally in a private temporary directory. The implemented workflow is URL analysis, user-selected offered format/quality, job creation, background processing, progress polling, and owner-authenticated retrieval.

The current implementation is a functional development-stage application, not a production deployment. Automated backend checks pass, but the repository contains no evidence of a complete live external-media download test. yt-dlp is optional and disabled by the supplied backend example configuration.

```
flowchart TD
    U[User] --> F[React frontend]
    F -->|Analyze URL| API[FastAPI REST API]
    F -->|Create job with bearer token| API
    API --> AUTH[JWT authentication and ownership checks]
    API --> VAL[Provider URL and option validation]
    VAL --> REG[ProviderRegistry]
    REG --> DIR[DirectFileProvider]
    REG --> YTD[YtDlpProvider, opt-in]
    DIR --> JOB[PostgreSQL download job]
    YTD --> JOB
    JOB --> Q[FastAPI in-process background task]
    Q --> WORKER[Download worker]
    WORKER --> MEDIA[Direct HTTP or yt-dlp]
    MEDIA --> FF[FFmpeg when merging is required]
    WORKER --> PROBE[ffprobe validation]
    PROBE --> STORE[Private temporary local storage]
    STORE --> FILE[Owner-authenticated file endpoint]
    FILE --> F
```

The diagram describes the implementation rather than a guarantee that every source will be extractable. The application does not maintain one downloader per platform. yt-dlp is used as a general media extraction/download engine for URLs it legitimately supports, subject to its availability, the configured opt-in, source behavior, and applicable rules.

2. Technology Stack

Area

Verified technology

Frontend

React 19, TypeScript 6, Vite 8, Tailwind CSS 4, lucide-react

Frontend checks

Oxlint; TypeScript project build via `tsc -b`; Vite production build

Backend

Python (no project-declared minimum), FastAPI, Pydantic 2, pydantic-settings, SQLAlchemy 2, Alembic

Database

PostgreSQL 17 Alpine in Compose; psycopg 3 driver

Media

Optional yt-dlp, FFmpeg executable for stream merging, ffprobe executable for output validation

Authentication

PyJWT, `pwdlib[argon2]` recommended password hasher (verified test hashes use Argon2id)

Testing

pytest, FastAPI TestClient, SQLite in-memory test database

Infrastructure

Docker Compose provisions PostgreSQL only; Vite and Uvicorn are run separately

Frontend dependency ranges are in package.json and the lockfile. Backend dependencies are pinned in requirements.txt; optional yt-dlp is pinned separately in requirements-ytdlp.txt. There is no `pyproject.toml` or frontend test script in the inspected repository.

3. Repository Structure

```
.
├── docs/                          # Project and audit documentation
├── public/                        # Vite public assets
├── src/
│   ├── assets/                    # Frontend assets
│   ├── components/                # Landing, navigation, result, and
layout UI
│   ├── context/                   # React authentication context/
provider
│   ├── lib/                       # API client and session token
storage
│   ├── pages/                     # Home, login, registration, history
│   └── types/                     # API response/request TypeScript
types
```

```
├── backend/
│   ├── alembic/                    # Migration environment and revisions
│   ├── app/
│   │   ├── api/                    # FastAPI dependencies and route
modules
│   │   ├── core/                   # Settings, database, network guard
│   │   ├── models/                 # SQLAlchemy User and Download models
│   │   ├── providers/              # Provider protocol, registry,
implementations
│   │   ├── schemas/                # Pydantic API contracts
│   │   ├── services/               # Auth, jobs, storage, cleanup,
analysis
│   │   └── workers/                # Download worker and validation
│   ├── storage/tmp/                # Default private temporary media
root
│   ├── tests/                      # Backend tests; external providers
are mocked
│   ├── requirements.txt            # Pinned base backend dependencies
│   └── requirements-ytdlp.txt      # Optional yt-dlp dependency
├── docker-compose.yml              # PostgreSQL-only local service
├── package.json                    # Frontend scripts and dependencies
├── .env.example                    # Frontend API base URL example
└── backend/.env.example            # Backend and Compose safe
placeholders
```

Generated output, installed dependencies, virtual environments, caches, and local storage contents are intentionally omitted. Notable entry points are src/main.tsx, src/App.tsx, backend/app/main.py, and backend/alembic/env.py.

4. Frontend Architecture

- `src/main.tsx` mounts `App` under React `StrictMode`. `App.tsx` provides auth context and selects pages using `window.location.pathname`; there is no routing library.
- `/` renders the landing page and URL analyzer. `/login` and `/register` render account forms. `/history` checks auth and redirects unauthenticated users to `/login`.
- `AuthProvider` restores the session by validating the saved token through `/api/auth/me`. Login stores the access token; logout removes it client-side. Registration creates an account but does not automatically sign in.
- `authStorage.ts` uses `sessionStorage` with key `downloadit_access_token`. Authenticated calls attach `Authorization: Bearer ...` in `src/lib/api.ts`.
- URL analysis is anonymous. The client performs basic URL parsing, then calls `POST /api/media/analyze`. The API returns platform and extractor metadata, but the current result component displays the platform label rather than the separate extractor field; only formats returned by the backend are selectable.
- The result selector is populated from `available_formats`, so the UI does not expose arbitrary yt-dlp selectors. yt-dlp results require an explicit authorization checkbox before job creation.
- Job creation, status polling (initial delay about one second, then 1.5-second intervals),

completion/error handling, and authenticated file retrieval are in `UrlDownloader.tsx`. The downloaded blob is saved with a generated `DownloadIt-<job-id>` name.

`History.tsx` fetches the signed-in user's list once per mount, displays title/source/platform/status/format/quality/created time, and offers download for completed records. It does not continuously poll or automatically refresh status.

State is local React state plus a single context for authentication; there is no global query/store library. TypeScript contracts are in `src/types/`.

Tailwind responsive classes provide responsive layouts, mobile navigation, and horizontal overflow for the history table. Automated browser/mobile testing is not configured.

Major files: UrlDownloader.tsx, MockResult.tsx (the name is historical; it renders real analysis responses), History.tsx, api.ts, auth-provider.tsx, download.ts, and media.ts.

5. Backend Architecture

`app.main` constructs the FastAPI application, configures CORS, registers exception handling and route modules, and exposes `/`. Configuration comes from Pydantic Settings and `backend/.env`; SQLAlchemy creates a PostgreSQL engine and request-scoped sessions through `get_db`.

Route handlers validate request/response schemas and delegate work to services. Authentication is provided by a bearer-token dependency. Media analysis selects and calls a provider. Download creation persists a pending row and enqueues a FastAPI `BackgroundTasks` callback; the worker reopens a database session, updates status/progress, invokes the selected provider, validates output with ffprobe, and records completion or a sanitized failure. Files are served through a separate authenticated route.

```
sequenceDiagram
    participant Browser
    participant API as FastAPI route
    participant Auth as Auth/ownership dependency
    participant DB as PostgreSQL
    participant Service as Download service
    participant Worker as Background worker
    participant Provider as Selected provider
    participant Storage as Temporary storage
    Browser->>API: POST /api/downloads + bearer token
    API->>Auth: Validate JWT and active user
    Auth->>DB: Load user
    API->>Service: Validate provider/options/acknowledgement
    Service->>DB: Insert pending job with user_id
    API-->>Browser: 201 {id, status: pending}
    API->>Worker: Enqueue in-process BackgroundTask
    Worker->>DB: pending -> processing
    Worker->>Provider: Analyze and download with progress callback
    Provider->>Storage: Write generated temporary file
    Worker->>DB: Store progress then completed/failed
    Browser->>API: GET /api/downloads/{id}/status
    API->>Auth: Validate token and job ownership
```

```
    API-->>Browser: Current status/progress
    Browser->>API: GET /api/downloads/{id}/file
    API->>Auth: Validate token and job ownership
    API-->>Browser: FileResponse for completed, unexpired file
```

The queue is process-local and not durable. There is no scheduler/startup hook for cleanup. See ARCHITECTURE.md for diagrams and boundaries.

6. API Documentation

There are **11 `/api` routes** and one root `GET /` route. Swagger UI and ReDoc are exposed by FastAPI at `/docs` and `/redoc` when the server is running.

Method and route

Auth

Request and response

Behavior and common errors

`GET /`

No

No body; `{service, version}`

Service identity.

`GET /api/health`

No

No body; `{status, service, database}`

Performs `SELECT 1`; `503` when DB is unavailable.

`POST /api/auth/register`

No

`{email, password}` -> user `{id,email,created_at,is_active}`

Password policy enforced; `201`, `409` duplicate email, `422` invalid data.

`POST /api/auth/login`

No

`{email,password}` -> `{access_token,token_type}`

`401` for bad credentials; response uses Bearer token.

`GET /api/auth/me`

Bearer

No body -> user object

`401` missing/invalid/expired token or inactive/missing user.

`POST /api/media/analyze`

No

`{url}` -> analysis metadata and `available_formats[]`

HTTP(S) syntax is checked; provider errors map to `400`, `422`, or `503`; unexpected errors are sanitized as `502`.

`GET /api/downloads`

Bearer

No body -> array of history items

Owner-filtered and newest first. `401` without valid token.

`POST /api/downloads`

Bearer

`{source_url,format?,quality?,authorized_to_download?}` -> `{id,status}`

`201` pending; provider options and authorization acknowledgement checked; errors may be `400`, `401`, `422`, `503`.

`GET /api/downloads/{download_id}`

Bearer

UUID path -> detail

Owner-scoped; foreign/unknown ID returns `404`; invalid UUID `422`.

`GET /api/downloads/{download_id}/status`

Bearer

UUID path -> `{id,status,progress,downloaded_bytes,total_bytes,error}`

Owner-scoped; checks and applies expiration; missing/foreign ID `404`.

`GET /api/downloads/{download_id}/file`

Bearer

UUID path -> attachment

Owner-scoped; completed/unexpired only; `404` missing/foreign/file absent, `409` not ready, `410` expired.

`DELETE /api/downloads/{download_id}`

Bearer

UUID path -> `{success:true}`

Owner-scoped; processing job `409`; removes completed storage and row; foreign/unknown `404`.

`MediaAnalyzeResponse` includes `success`, `platform`, `extractor`, `requires_authorization`, optional title/duration/thumbnail/webpage/uploader, and `available_formats`. A format includes output format, quality, extension, resolution, estimated file size, and codec metadata. Download history omits the raw error and storage path; detail adds owner ID, expiration, and error field.

For field-level contracts and response details see API.md; schemas are implemented under `backend/app/schemas/` and routes under `backend/app/api/routes/`.

Existing documentation discrepancy: `backend/README.md` describes media analysis as authenticated. The current `POST /api/media/analyze` route has no authentication dependency, so analysis is public; job creation, history, status, detail, delete, and file access require a bearer token.

7. Authentication

Registration normalizes email and requires at least 12 characters including uppercase, lowercase, a digit, and a symbol. `pwdlib.PasswordHash.recommended()` hashes passwords; tests verify Argon2id-form hashes. Login returns a signed HS256-by-default JWT

containing the subject user UUID and expiry. Default/example lifetime is 30 minutes; configuration caps it at 1,440 minutes. `/api/auth/me` validates signature/expiry and reloads an active user from the database.

The frontend stores the bearer token in `sessionStorage`. It is scoped to the tab session and removed at logout, but it remains readable to JavaScript in that origin; an XSS compromise could steal it. There is no refresh-token flow, server-side logout/revocation, or httpOnly-cookie strategy. HTTPS and appropriate production frontend protections remain deployment responsibilities. Logout is client-side token removal.

8. Authorization and IDOR Protection

At job creation, `user_id` is set from the authenticated user, not from request data. Read, status, file, delete, and history queries constrain records to that user. A job belonging to another account is returned as `404 Download not found`, intentionally matching a nonexistent ID. This protects job details, history, and files; file retrieval also requires completed/unexpired status and a validated storage-relative path.

```
User A creates job -> downloads.user_id = A
User B requests job or file -> query matches (job_id AND user_id = B)
No matching row -> 404; file response is not reached
```

This behavior is covered by auth/download tests. See SECURITY.md.

9. Database Design

The two tables are `users` and `downloads`. UUID primary keys are generated in application code. `downloads.user_id` references `users.id` with `ON DELETE CASCADE`; ORM relationships also cascade delete-orphan. Download status is constrained to `pending`, `processing`, `completed`, `failed`, or `expired`.

```
erDiagram
    USERS ||--o{ DOWNLOADS : owns
    USERS {
        uuid id PK
        varchar email UK
        varchar password_hash
        boolean is_active
        datetime created_at
        datetime updated_at
    }
    DOWNLOADS {
        uuid id PK
        uuid user_id FK
        varchar source_url
        varchar platform
        varchar title
        varchar status
        varchar format
        varchar quality
```

```
        varchar file_path
        datetime created_at
        datetime completed_at
        datetime expires_at
        text error_message
        bigint downloaded_bytes
        bigint total_bytes
        integer progress_percent
    }
```

Migration sequence: `0001_initial` creates user/download history; `0002_user_auth` adds password hash/active state, normalizes existing email and adds case-insensitive uniqueness; `0003_download_progress` adds byte counters and percentage. The verified Alembic head is `0003_download_progress`. Schema detail is in DATABASE.md.

10. Download Job Lifecycle

```
stateDiagram-v2
    [*] --> pending: create job
    pending --> processing: worker starts
    processing --> completed: download and ffprobe validation pass
    processing --> failed: provider, size, or validation failure
    completed --> expired: expiry observed by status/file access or
cleanup invocation
    pending --> [*]: user deletes job
    failed --> [*]: user deletes job
    expired --> [*]: user deletes job
    completed --> [*]: user deletes job
```

Creation validates the selected provider and format/quality, then commits a `pending` row. The worker marks it `processing`, analyzes again, selects a server-side format plan, downloads, checks file/path/size/partial-file conditions, validates media with ffprobe, then marks `completed`. Completed files get `completed_at` and `expires_at`; failures clear the file path and store a generic public error, except selected safe FFmpeg/quality/provider messages. Progress is periodically persisted. `cleanup_temporary_storage()` expires records and removes orphan files, but no scheduled caller is present. Status and file access also perform expiration checks. The history list itself does not.

11. Provider Architecture

`MediaProvider` is a structural protocol defining enablement, URL validation, options, analysis, selection, and download. `ProviderRegistry` checks providers in configured order and chooses the first enabled matching provider. The default order is `DirectFileProvider`, then `YtDlpProvider`.

DirectFileProvider. Accepts only HTTPS URLs with a path ending `.mp4`, default/443 port, no embedded credentials, and a hostname exactly present in `DOWNLOAD_ALLOWED_HOSTS`. Analysis makes a `HEAD` request; only `video/mp4` is accepted. Both analysis and streaming download reject redirects. Download is streamed in chunks, bounded by

DOWNLOAD_MAX_BYTES, and does not trust the source filename. Its sole selection is mp4/source.

YtDlpProvider. Enabled only when YTDLP_ENABLED=true and optional yt-dlp is installed. It attempts safe public HTTP(S) URLs using yt-dlp's extractor behavior. Actual extractor/platform data is reported after metadata extraction; the provider does not claim a fixed comprehensive set of platforms. It requires the per-job authorization acknowledgement, disables playlists, accepts no user-supplied yt-dlp options/cookies/credentials/custom headers, and sets yt-dlp's proxy option to empty. Media is limited to generated MP4 or M4A output plans and controlled quality labels. FFmpeg is required for separate stream merging.

Scope statement: The application does NOT maintain separate downloader implementations for every platform. It delegates supported media extraction/download to yt-dlp when YtDlpProvider is enabled and yt-dlp legitimately supports the supplied URL. DownloadIt does not promise support for every website or extractor.

12. yt-dlp Integration

YtDlpProvider imports YoutubeDL lazily from the optional package. Analysis invokes extract_info(url, download=False) under the public-network DNS guard with quiet logging, noplaylist, no cache, empty proxy, bounded timeouts/retries, and no cookie or arbitrary-header options. Playlist/multi-video results are rejected. Extractor key/name is mapped to a lowercase platform label dynamically.

Available formats are derived only from MP4 video streams and M4A audio streams. Server-owned _YtDlpPlan objects hold selector strings; the client supplies only format/quality labels. Download config uses a UUID-based output template, max_filesize, a progress hook, no overwrite, and yt-dlp partial-file handling. noplaylist=True is applied to both analysis and download. When needed, FFmpeg merges compatible separate video/audio streams into MP4. Provider errors are classified and translated to safe messages; raw extractor diagnostics are not returned.

13. Quality Selection

The frontend renders exactly the formats returned for the analyzed item. yt-dlp's controlled labels are MP4 best, 1080p, 720p, 480p, 360p, and M4A audio, but each is offered only when a compatible format is reported and within the known-size limit. An unlisted request is rejected; a listed choice may still become unavailable between analysis and download. Direct MP4 offers only mp4/source. The create-job endpoint receives the requested format/quality but validates it server-side; it does not accept selector expressions.

14. FFmpeg and ffprobe

FFmpeg is looked up on PATH. yt-dlp uses it only when selected output needs a merge; otherwise a single compatible stream can be downloaded directly. The direct-file provider does not invoke FFmpeg. ffprobe is required for every worker-completed file. It runs with a 15-second timeout and JSON output, checking container and stream types: MP4 must report an mp4 format token and a video stream; M4A must report an m4a token, an audio stream, and no video stream. Missing ffprobe, nonzero exit, malformed output, unexpected container, or missing required

stream fails the job. The worker also rejects empty/missing/symlink destination files, over-limit files, and residual sibling files.

15. File Storage

Default storage is `backend/storage/tmp` (configurable with `DOWNLOAD_STORAGE_DIR`). Each job receives a UUID directory and a random UUID-like filename with `.mp4` or `.m4a`; database paths are relative. Root/job directories are set to `0700`, completed files to `0600`. Storage resolution rejects absolute paths, `..`, unexpected path depth, invalid generated UUID components/extensions, and paths resolving outside the root. Storage is local and temporary; it is not in PostgreSQL and is not exposed as a static web directory. The file endpoint streams only an authenticated owner's completed and unexpired file. Default maximum file size and lifetime are 50 MiB and 30 minutes.

16. Security Architecture

Threat

Implemented protection

Code location

Unauthorized account/job access

JWT bearer dependency, active-user lookup, owner-filtered queries

`backend/app/api/dependencies.py`, `services/download_service.py`, route modules

Password disclosure

Recommended password hashing; hashes excluded from response schemas

`services/auth_service.py`, `schemas/auth.py`

IDOR/file theft

Job queries include both ID and owner ID; file route requires auth

`api/routes/downloads.py`, `services/download_service.py`

SSRF to local/private destinations

URL scheme/host/port checks, DNS/IP public-address checks, connection-time DNS guard

`providers/ytdlp_provider.py`, `providers/direct_file.py`, `core/network_security.py`

Redirect-to-internal destination

Direct provider disables redirects and rejects 3xx; yt-dlp requests are guarded at connection resolution

Provider implementations

Proxy-based destination bypass

yt-dlp gets `proxy=''`; direct provider uses default httpx `trust_env` behavior (see SSRF caveat)

Provider implementations

Resource exhaustion via large media

Advertised-size filtering, yt-dlp max file size, streaming/progress/final size checks

Provider modules, worker

Arbitrary output path/name

Generated job/file UUIDs, restricted extensions, containment checks

`services/storage_service.py`

Partial/misleading output

Symlink/empty/residual-file checks and ffprobe container/stream verification

`workers/download_worker.py`

Error detail disclosure

Provider mapping, generic worker failure, safe DB exception handler

`providers/errors.py`, `workers/download_worker.py`, `app/main.py`

Unauthorized extractor options or playlist processing

Controlled selector mapping, no client yt-dlp options, `noplaylist=True`

`providers/ytdlp_provider.py`

Stale media retention

Expiry timestamp, lazy expiry on status/file access, explicit cleanup service

`services/download_service.py`, `services/storage_cleanup.py`

No DRM bypass, authentication bypass, cookie/session acquisition, CAPTCHA bypass, rate-limit bypass, or private-content extraction is implemented or recommended. The explicit authorization checkbox is user acknowledgement, not a rights verification mechanism.

17. SSRF Protection

For yt-dlp URLs the sequence is: parse HTTP(S), reject credentials/local suffixes/disallowed ports, reject invalid host forms, resolve all returned addresses, require every address to be global, and reject playlist paths. The request then runs inside `public_network_dns_guard()`, which checks every `socket.getaddrinfo` result during the guarded operation and rejects any non-global address. The guard is applied around yt-dlp analysis/download and direct HTTP calls; direct URLs are additionally constrained to exact host allowlist and HTTPS.

Checking only before a request can be insufficient because DNS can change between validation and connection. The connection-time guard narrows that DNS-rebinding window by checking resolution at lookup time too. Direct provider has redirects disabled. yt-dlp may follow extractor-managed URLs, with guarded socket resolution. yt-dlp explicitly disables proxy use. In contrast, `httpx.Client` for direct MP4 does not set `trust_env=False`; its default may honor environment proxy variables. The DNS guard does not independently establish the destination used by an external proxy, so deployments must account for proxy environment configuration. This is an implementation limitation, not a claim that proxy-mediated DNS is guarded.

Only yt-dlp rejects all private/non-global resolutions generically. Direct provider's exact host allowlist and guarded resolution are its controls. HTTP is accepted for yt-dlp but not the direct-file provider.

18. Error Handling

Provider exceptions map to stable public messages: malformed URL `422`; unsupported source and unsuitable content generally `400`; unavailable provider/FFmpeg `503`; unavailable quality `400`; processing/analysis fallback `502`. Authentication errors use `401`; foreign/missing jobs use `404`; pending/processing file retrieval uses `409`; expired file retrieval uses `410`. Worker errors persist generic text, with selected safe messages for missing FFmpeg, unavailable quality, and temporary provider failure. SQLAlchemy errors become `503` without exposing exception details. Frontend API helpers extract safe `detail` values where provided and otherwise use generic connection/service messages.

19. Download Progress

The `downloads` table stores nullable `downloaded_bytes`, `total_bytes`, and `progress_percent`. Provider callbacks report bytes and optional total. The worker refuses progress past the maximum and persists updates no more than once per second (plus the first

report). Percentage is integer `downloaded * 100 / total`, capped at 100; it stays null without a positive total. Completion sets byte fields to final size and progress to 100. `GET .../status` returns these fields. The frontend polls while status is pending/processing and displays percentage/byte counts when known; failed/completed/expired stop polling.

20. History

History rows are stored per user and queried newest-first. The UI displays title (or fallback), source URL, platform label, status, output format, quality, and creation time. Completed rows expose the authenticated file action; pending/processing and terminal states are labeled. The list does not expose backend error details or expiration timestamp. It does not poll status. Since lazy expiry is applied by status/file access and the cleanup service is not scheduled, a history row can continue to appear completed after its file lifetime until one of those paths observes expiration.

21. Configuration

Variable

Required?

Default/example

Purpose

Sensitive?

`DATABASE_URL`

Yes

Example uses local PostgreSQL URL

SQLAlchemy connection; must match Compose credentials/host

Yes

`JWT_SECRET_KEY`

Yes

Safe placeholder in example

HMAC signing key

Yes

`JWT_ALGORITHM`

No

`HS256`

JWT signing/verification algorithm

No

`ACCESS_TOKEN_EXPIRE_MINUTES`

No

`30`

Token lifetime, range 1–1440

No

`APP_ENV`

No

`development`

Settings value; no behavior switch was found

No

`APP_NAME`

No

`DownloadIt API`

Service identity

No

`APP_VERSION`

No

`1.0.0`

Service version

No

`FRONTEND_URL`

No

`http://localhost:5173`

CORS configured origin

No

`DOWNLOAD_ALLOWED_HOSTS`

No

Example: `samplelib.com`

Exact host allowlist for direct MP4

No

`DOWNLOAD_FILE_EXPIRATION_MINUTES`

No

`30`

Completed media lifetime, range 1–10080

No

`DOWNLOAD_MAX_BYTES`

No

`52428800`

Media cap in bytes (50 MiB default)

No

`DOWNLOAD_STORAGE_DIR`

No

`backend/storage/tmp`

Local temporary storage root

No

`YTDLP_ENABLED`

No

`false`

Opt in to generic yt-dlp provider

No

`POSTGRES_DB`

Compose-required

Example: `downloadit`

PostgreSQL container database name

No

`POSTGRES_USER`

Compose-required

Example: `downloadit`

PostgreSQL container user

Yes

`POSTGRES_PASSWORD`

Compose-required

Replace placeholder locally

PostgreSQL container password

Yes

VITE_API_BASE_URL

No

http://localhost:8000

Frontend API origin

No

Backend settings read backend/.env; only the values in .env.example are safe templates. POSTGRES_* must be explicitly present for Compose and credentials must be consistent with DATABASE_URL. APP_ENV is defined but no runtime mode behavior was observed. Do not put production secrets in committed example files.

22. Docker and Local Infrastructure

docker-compose.yml defines one postgres:17-alpine service. It publishes only on loopback (127.0.0.1:5432), stores database files in named volume downloadit-postgres-data, restarts unless stopped, and checks readiness with pg_isready. Compose requires POSTGRES_DB, POSTGRES_USER, and POSTGRES_PASSWORD; use docker compose --env-file backend/.env up -d. It does not define frontend, backend, worker, or migration containers and has no application dependency/startup ordering. Apply migrations and start API/UI separately.

23. Local Development Setup

Prerequisites: macOS, Node.js/npm compatible with Vite 8, Python with venv/pip, Docker Desktop or local PostgreSQL, and (for media validation) FFmpeg/ffprobe. Vite 8's supported Node floor should be confirmed against the installed runtime before deployment; no .nvmrc is present.

```
# From the repository root
cp backend/.env.example backend/.env
# Edit backend/.env: set a strong random JWT_SECRET_KEY and matching
local DB values.
cp .env.example .env

brew install ffmpeg
docker compose --env-file backend/.env up -d

cd backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
# Optional only when authorized/permitted yt-dlp testing is needed:
```

```
pip install -r requirements-ytdlp.txt
alembic upgrade head
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Run the frontend in a second terminal from the repository root:

```
npm install
npm run dev
```

Open the Vite URL (normally `http://localhost:5173`). Set `YTDLP_ENABLED=true` only after installing the optional dependency and reviewing the legal/security scope. The backend example defaults to `false`. Never reuse placeholder JWT/database credentials outside local development. Detailed instructions are in SETUP.md.

24. Running Tests

```
cd backend
source .venv/bin/activate
pytest
pytest tests/test_ytdlp_provider.py
```

Frontend commands from repository root:

```
npm run lint
npm run build
```

There is no frontend unit/e2e test script. Backend tests use in-memory SQLite, replace the queue with a no-op, and mock external provider/network behavior. They verify contracts and security logic, but do not prove current live extractor/site behavior. Current validation during this documentation task: **92 passed**, one Starlette TestClient deprecation warning; frontend lint completed cleanly; frontend build succeeded; Alembic reports `0003_download_progress` as head. See TESTING.md.

25. Verified End-to-End Behavior

Repository evidence: unit/integration tests cover account registration/login, authenticated job creation/history/status, separate worker processing, fake-provider progress, output validation (with mocked ffprobe), authenticated owner file access, and cleanup invocation. Provider and yt-dlp tests use fake HTTP clients/FakeYoutubeDL and mocked DNS. URLs that resemble public sample, Instagram, and YouTube URLs are fixture inputs, not proof of requests to those sites.

Live-tested: no repository artifact, test report, or source evidence was found demonstrating a real external media analysis-to-download flow. During this documentation task, no live external media was requested or downloaded. Do not treat the fixture URLs as historical live-test evidence. The application can only claim the behavior covered by its source and automated tests, not current site availability.

26. Test Coverage

The 92 backend tests cover authentication and token validation; ownership/IDOR and API history; health/database error handling; URL validation and provider selection; direct-provider

MIME/redirect/error behavior; yt-dlp enablement, metadata mapping, playlist rejection, public/private address checks, controlled quality, size limit, progress, and error mapping; worker state/progress/size/failure; ffprobe validation; storage path constraints; file access/expiry; and cleanup logic. Test files: `test_auth.py`, `test_downloads.py`, `test_health.py`, `test_providers.py`, `test_ytdlp_provider.py`, and `test_download_worker.py`.

Gaps: live upstream integration, frontend automated tests, browser/mobile accessibility checks, multi-process queue behavior, actual PostgreSQL migration application in the test suite, and an automatically scheduled cleanup path are not verified by this test suite.

27. Frontend UX

Current screens include a landing page with URL input and feature copy, analysis/loading/error states, metadata preview, detected platform label (not the separate extractor field), duration/uploader, available quality selector, authorization acknowledgement when required, job progress/completed/failure/expired states, login/register, responsive navigation, and a history table. No separate “platform catalog” is guaranteed; the provider strip currently lists Direct MP4 only.

Some visible polish/product elements are placeholders: navigation items for Features/Platforms/Pricing and footer Privacy/Terms use `href="#"`; the features section contains broad marketing claims not enforced as service-level guarantees. The result component’s `MockResult` name is misleading but it consumes API results. These are documented as incomplete, not silently represented as implemented workflows.

28. Current Limitations

- No claim that every website is supported; yt-dlp extractor compatibility and source policies change.
- yt-dlp is optional and example-disabled; direct MP4 is restricted to exact allowlisted hosts.
- Playlists are rejected. Authentication-required/private content is not bypassed. DRM/access controls are not bypassed.
- Default file cap is 50 MiB; storage is local and ephemeral, not shared/object storage.
- Queue is FastAPI in-process `BackgroundTasks`, without durable retries, external queue, or worker coordination.
- Cleanup is implemented as a service but is not automatically scheduled; history may remain stale until expiry is observed.
- Session token in `sessionStorage` is accessible to same-origin JavaScript; no refresh or revocation mechanism exists.
- Direct provider’s httpx client may honor proxy environment variables; yt-dlp disables proxies. Proxy-mediated resolution is not independently guarded by the DNS wrapper.
- Existing backend README calls media analysis authenticated, but the current analysis route is public. Its no-proxy wording applies to yt-dlp options, not the default httpx behavior of DirectFileProvider. The ignored local `.env` was not inspected, so its actual yt-dlp enablement was not verified.
- No rate limiting, production deployment manifests, monitoring, backups, or structured production logging were found.
- Frontend pages and components have no automated test suite; placeholders remain in navigation/footer.

29. Production Readiness Assessment

Area

Current state

Production ready?

Remaining work

Frontend

Working Vite app; lint/build pass

Partial

Real UX/accessibility/browser review; replace placeholder links; add tests.

Backend/API

FastAPI routes/services present

Partial

Deployment config, operational hardening, rate limits, observability.

Database

PostgreSQL schema/migrations present

Partial

Production provisioning, backup/restore, migration operations.

Authentication

Argon2id-style hashing, short JWT

Partial

Secure production token lifecycle/storage and abuse controls.

Authorization

Owner filters on job/file operations

	Yes for implemented endpoints
Retain regression tests; audit any new routes.	
Downloader	
Direct HTTP and optional yt-dlp	
	Partial
Authorized live source validation, operational limits, durable execution.	
Storage	
Private local temp files and containment	
	Partial
Shared/durable storage policy, automated cleanup, capacity monitoring.	
SSRF	
Public resolution checks and connection guard	
	Partial
Resolve proxy caveat; deployment-specific egress controls and review.	
Secrets	
SecretStr and ignored env files	
	Partial
Managed secret injection/rotation; ensure no placeholder deployment.	
CORS	
Configured origin plus local development origins/regex	
	No for production unchanged
Narrow allowed origins/methods/headers for deployed environment.	
Rate limiting	

Not implemented

No

Add abuse controls before public exposure.

Logging/monitoring

No application observability stack found

No

Structured logs, metrics, tracing, alerting.

Job queue/concurrency

In-process FastAPI background task

No

Durable queue, worker isolation, retries/idempotency.

Deployment/HTTPS

No app deployment configuration found

No

TLS termination, hardened deployment, secret/egress controls.

Backups

Not configured

No

Database/storage backup and recovery procedures.

Cleanup

Expiry and callable cleanup service

Partial

Schedule cleanup and monitor failures/storage use.

30. Development History

No `.git` metadata or commit history is present in the supplied workspace. The three Alembic revisions establish only this schema sequence: initial users/download history (`0001_initial`), authentication fields and normalized email uniqueness (`0002_user_auth`), then progress counters (`0003_download_progress`). Current source proves that direct provider, auth, provider registry, optional yt-dlp, progress, media validation, DNS guard, storage cleanup, and tests exist, but does not establish when or in which numbered product phase they were added.

Exact implementation history could not be fully reconstructed from repository evidence. Phases 1–12 cannot be truthfully assigned beyond the migration sequence above. Historical live validation of Instagram/YouTube or other providers was not evidenced in repository artifacts inspected for this audit.

31. Current Project Status

Area

Status

Evidence

Frontend

PARTIAL

Pages/components implemented; placeholders and no frontend test suite.

Backend

COMPLETE for current local API scope

Routes/services implemented; operational production features absent.

Database

COMPLETE for current schema

Two tables, three migrations, current head verified.

Authentication

COMPLETE for basic account/JWT flow

Registration/login/me and tests; production token caveats remain.

Downloader

PARTIAL

Direct file and optional yt-dlp providers; no live E2E evidence.

Providers

COMPLETE for two current implementations

DirectFileProvider and YtDlpProvider; latter opt-in.

Media validation

COMPLETE in worker path

ffprobe required, size/container/stream/residual checks tested.

Security

PARTIAL

Ownership, JWT, URL/DNS/storage controls exist; gaps in production hardening.

Testing

COMPLETE for configured backend validation

92 passed; tests are mostly isolated/mocked.

Infrastructure

PARTIAL

Compose runs PostgreSQL only.

Documentation

COMPLETE after this documentation set

Master plus focused references and audit report.

Deployment readiness

NOT STARTED

No production deployment configuration/operational stack found.

yt-dlp is **OPTIONAL/DISABLED** by the provided example. Automated tests are **TESTED**; live external integration is **NOT VERIFIED**. Cleanup is implemented but automatic scheduling is **NOT IMPLEMENTED**.

32. Recommended Next Phases

Phase 13: UI/UX polish and real-user testing. Resolve placeholder navigation/footer links, review claims against service behavior, add browser/mobile/accessibility checks, and evaluate sign-in-to-download ergonomics.

Phase 14: Production deployment and hardening. Design a durable queue/worker, scheduled cleanup, rate limits, production token strategy, deployment-managed secrets, narrow CORS, HTTPS/egress policy, logging/monitoring, storage capacity/retention, and database backups. Validate only sources the operator and user are authorized to use. These are recommendations only; no implementation changes were made.

Further detail: PRODUCTION_READINESS.md.

Architecture Reference

This document expands the system and request flow described in PROJECT_DOCUMENTATION.md. Source references are relative to the repository root.

Components and Boundaries

Component

Responsibility

Source

React client

Pathname-based page selection, auth context, URL analysis, job polling, history/file UI

`src/App.tsx`, `src/components/UrlDownloader.tsx`, `src/lib/api.ts`

FastAPI application

CORS, router mounting, database error handling

`backend/app/main.py`

API routers

HTTP validation, auth dependency injection, response construction

`backend/app/api/routes/`

Services

User/job lifecycle, media analysis, expiry, storage cleanup

`backend/app/services/`

Provider registry

Enabled first-match provider selection

`backend/app/providers/registry.py`

Worker

Process pending job, persist progress/state, validate output

`backend/app/workers/download_worker.py`

Database

User/job metadata and progress

`backend/app/models/`, `backend/alembic/`

Temporary storage

Generated local file paths and permissions

`backend/app/services/storage_service.py`

System Architecture

```
flowchart LR
    Browser[React + TypeScript] -->|HTTP JSON / bearer| API[FastAPI]
    API --> Auth[JWT user dependency]
    API --> Services[Service layer]
    Services --> DB[(PostgreSQL)]
    Services --> Registry[ProviderRegistry]
    Registry --> Direct[DirectFileProvider]
    Registry --> Ytdlp[YtDlpProvider opt-in]
```

```
    API --> Queue[FastAPI BackgroundTasks]
    Queue --> Worker[Download worker]
    Worker --> Direct
    Worker --> Ytdlp
    Direct --> Remote[Allowlisted HTTPS MP4]
    Ytdlp --> Remote2[Public HTTP(S) URL via yt-dlp]
    Worker --> Probe[ffprobe validation]
    Ytdlp --> FFmpeg[FFmpeg for stream merge]
    Worker --> Files[(Private local temporary storage)]
    Browser -->|authenticated status/file| API
    API --> Files
```

Media analysis is not inherently an authenticated route. Job creation, history, job detail/status, deletion, and file retrieval are authenticated.

Download Request Lifecycle

```
sequenceDiagram
    actor User
    participant UI as React client
    participant API as FastAPI
    participant DB as PostgreSQL
    participant Worker
    participant Provider
    participant Disk as Temporary storage
    User->>UI: Submit URL
    UI->>API: POST /api/media/analyze
    API->>Provider: Registry selection and analyze
    Provider-->>UI: Metadata and offered formats
    User->>UI: Select offered format; acknowledge if required
    UI->>API: POST /api/downloads with bearer token
    API->>DB: Insert pending owner-scoped job
    API-->>UI: 201 pending + job UUID
    API->>Worker: FastAPI BackgroundTask
    Worker->>DB: processing and progress updates
    Worker->>Provider: Analyze and download
    Provider->>Disk: Write generated output
    Worker->>Worker: Size/path/ffprobe/residual checks
    Worker->>DB: completed + expiration or failed + safe error
    loop pending or processing
        UI->>API: GET /api/downloads/{id}/status
        API-->>UI: status and progress fields
    end
    UI->>API: GET /api/downloads/{id}/file + bearer token
    API-->>UI: Authenticated attachment
```

Provider Selection

```
flowchart TD
    Input[HTTP(S) URL] --> DirectEnabled{Direct provider enabled?}
```

```
    DirectEnabled -->|Yes| DirectMatch{Exact allowlisted HTTPS .mp4?}
    DirectMatch -->|Yes| Direct[DirectFileProvider]
    DirectMatch -->|No| YtdlpEnabled{YtDlp enabled?}
    DirectEnabled -->|No| YtdlpEnabled
    YtdlpEnabled -->|No| Unsupported[UnsupportedPlatform]
    YtdlpEnabled -->|Yes| YtdlpMatch{HTTP(S) hostname and safe public
resolution?}
    YtdlpMatch -->|No| Reject[Reject URL/provider]
    YtdlpMatch -->|Yes| Ytdlp[YtDlpProvider]
    Direct --> Analyze[Analyze or download]
    Ytdlp --> Extractor[yt-dlp extraction attempt]
    Extractor -->|recognized and valid output| Analyze
    Extractor -->|unsupported/error| Reject
```

The registry order in `default_provider_registry` is direct then yt-dlp. A match is not a support guarantee: provider analysis may reject a URL or report no supported formats.

Job State Machine

```
stateDiagram-v2
    [*] --> pending
    pending --> processing
    processing --> completed
    processing --> failed
    completed --> expired
    pending --> deleted
    processing --> deleted: not permitted
    completed --> deleted
    failed --> deleted
    expired --> deleted
```

`deleted` is not a persisted status; it represents row removal. Processing deletion returns `409`. Expiration is observed lazily by status/file endpoints or when `cleanup_temporary_storage()` is called; no scheduler invokes it.

Deployment Shape

Compose provisions only PostgreSQL. Vite, Uvicorn, migration execution, and the background callback are local processes. No containerized app/worker, external broker, web server/TLS termination, or production topology is defined. The in-process background approach has no durable queue/retry contract and should not be assumed safe for multi-process production scaling.

API Reference

Base path is `/api`, except root `GET /`. Interactive OpenAPI is available at `/docs` and `/redoc` when FastAPI is running. UUID parameters are standard UUIDs. JSON bodies reject schema-invalid data with `422`.

Authentication

Method

Path

Auth

Request

Success

Errors

POST

`/api/auth/register`

No

`{email: string, password: string}`

`201 {id,email,created_at,is_active}`

`409` duplicate; `422` invalid email/password

POST

`/api/auth/login`

No

`{email: string, password: string}`

`200 {access_token,token_type:"bearer"}`

`401` same message for unknown email and wrong password; `422` invalid body

GET

`/api/auth/me`

Bearer

None

`200 {id,email,created_at,is_active}`

`401` missing, invalid, expired, inactive, or unknown user

Registration password policy: minimum 12 characters, at least one uppercase, lowercase, digit, and non-alphanumeric character. Email is stripped/case-folded.

The backend README currently calls media analysis authenticated, but the current analysis route has no auth dependency; download operations remain protected.

Media Analysis

`POST /api/media/analyze`

Authentication is not required. Body: `{ "url": "https://..." }`. The schema requires non-empty HTTP(S) URL syntax; the chosen provider adds stricter validation. Success is `200` with:

```
{
  "success": true,
  "platform": "extractor-derived-or-direct",
  "extractor": "optional provider name",
  "requires_authorization": true,
  "title": "optional",
  "duration": "optional string",
  "thumbnail": "optional URL",
  "webpage_url": "optional URL",
  "uploader": "optional",
  "available_formats": [

{"format":"mp4","quality":"720p","extension":".mp4","resolution":"1280x
720","filesize":null,"video_codec":null,"audio_codec":null}
  ]
}
```

Invalid syntax is `422`; unsupported/unusable media generally `400`; provider/network failure `503`; unexpected processing errors are sanitized to `502`. Metadata values are provider-reported and may be absent.

Downloads and History

All endpoints below require `Authorization: Bearer <access_token>`.

Method

Path

Success

Important errors

GET

`/api/downloads`

`200` array of owner history items, newest first

`401`, database unavailable `503`

POST

`/api/downloads`

`201 {id,status:"pending"}`

`400` unsupported options/required acknowledgement; `401`; `422`; `503`

GET

`/api/downloads/{download_id}`

`200` detail

`404` unknown or other-owner job; `422` malformed UUID

GET

`/api/downloads/{download_id}/status`

`200 {id,status,progress,downloaded_bytes,total_bytes,error}`

`404`; may mark completed job expired

GET

`/api/downloads/{download_id}/file`

`200` attachment (`video/mp4` or `audio/mp4`)

`404` absent/foreign/missing file; `409` not completed; `410` expired

DELETE

`/api/downloads/{download_id}`

`200 {success:true}`

`404` unknown/foreign; `409` processing; `422` invalid UUID

Create request fields: `source_url` (required, max 2048), `format` (default `mp4`, 1-20 characters), `quality` (default `source`, 1-40 characters), and `authorized_to_download` (default false). Defaults are schema defaults; a provider may reject them. Direct MP4 requires `mp4/source`; yt-dlp uses only controlled offered pairs. Only yt-dlp jobs require the acknowledgement.

History item fields: `id`, `source_url`, `platform`, `title`, `status`, `format`, `quality`, `created_at`, `completed_at`. Detail adds `user_id`, `expires_at`, `error_message`. Neither returns the local file path. Status is one of `pending`, `processing`, `completed`, `failed`, `expired`.

Health and Root

Method

Path

Auth

Response

GET

`/api/health`

No

`200 {status:"ok",service,database:"connected"}` or `503` safe unavailable response

GET

`/`

No

`200 {service,version}`

Authorization Semantics

Job lookup is `WHERE downloads.id = :id AND downloads.user_id = :authenticated_user_id`. Foreign and nonexistent resources both return `404`. File access checks ownership before expiry/state/path validation. See SECURITY.md and route

source in `../backend/app/api/routes/`.

Database Reference

The application uses SQLAlchemy models and Alembic migrations. The test suite swaps PostgreSQL for an in-memory SQLite database; migration application against a live PostgreSQL service was not performed in this documentation audit. `alembic heads` reports `0003_download_progress` as the current head.

Entity Relationship

```
erDiagram
    USERS ||--o{ DOWNLOADS : owns
    USERS {
        uuid id PK
        varchar email UK
        varchar password_hash nullable
        boolean is_active
        datetime created_at
        datetime updated_at
    }
    DOWNLOADS {
        uuid id PK
        uuid user_id FK
        varchar source_url
        varchar platform
        varchar title nullable
        varchar status
        varchar format
        varchar quality
        varchar file_path nullable
        datetime created_at
        datetime completed_at nullable
        datetime expires_at nullable
        text error_message nullable
        bigint downloaded_bytes nullable
        bigint total_bytes nullable
        integer progress_percent nullable
    }
```

Tables

users

Column

Type/constraints

Meaning

`id`

UUID primary key

Application-generated user identifier

`email`

`varchar(320)`, non-null, unique/indexed

Normalized account email; a functional unique index enforces `lower(email)`

`password_hash`

nullable `varchar(255)`

Hashed password; raw password is never persisted

`is_active`

non-null boolean, default true

Authentication eligibility

`created_at`

non-null timestamp, server default now

Creation time

`updated_at`

non-null timestamp, server default now/on update

Last ORM update time

ORM relationship `downloads` has delete-orphan cascade. The foreign key also declares `ON DELETE CASCADE`.

`downloads`

Column

Type/constraints

Meaning

`id`

UUID primary key

Job ID and storage directory name

`user_id`

UUID, non-null FK to users, indexed

Owning account

`source_url`

non-null `varchar(2048)`

Submitted media URL

`platform`

non-null `varchar(40)`

`direct` or provider-derived label

`title`

nullable `varchar(500)`

Provider metadata title

`status`

non-null `varchar(20)` with check

`pending`, `processing`, `completed`, `failed`, or `expired`

`format`

non-null `varchar(20)`

Output format, currently MP4/M4A

`quality`

non-null `varchar(40)`

Controlled quality label

`file_path`

nullable `varchar(2048)`

Relative path under configured temp root; not a full machine path

`created_at`

non-null timezone timestamp, server default now

Job creation

`completed_at`

nullable timezone timestamp

Successful completion

`expires_at`

nullable timezone timestamp

Media expiry deadline

`error_message`

nullable text

Sanitized public-safe worker failure

`downloaded_bytes`

nullable bigint

Last reported bytes/current final size

`total_bytes`

nullable bigint

Expected total when known/final size on completion

`progress_percent`

nullable integer

Percent when total is known; 100 on completion

Media bytes are not stored in the database.

Migration History

Order

Revision

Parent

Change

1

`0001_initial`

none

Creates users and downloads, base fields, status check, owner foreign key/index.

2

`0002_user_auth`

`0001_initial`

Adds nullable `password_hash`, non-null `is_active`, normalizes emails, adds functional case-insensitive unique index.

3

`0003_download_progress`

`0002_user_auth`

Adds nullable downloaded/total bytes and progress percent.

Migration source is in `../backend/alembic/versions/`; runtime setup is in `../backend/alembic/env.py` and `../backend/alembic.ini`.

Lifecycle Semantics

Job creation assigns `user_id` from the authenticated user. Successful worker completion sets title, relative file path, timestamps, expiry, byte counts, and `progress_percent=100`. Failure clears `file_path`; expiration clears the file reference after removal attempt and changes status to `expired`. The file itself is local temporary storage and cleanup is separate from relational deletion.

Security Reference

Security controls documented here are present in source; they do not substitute for deployment review. See the master security overview.

Authentication Flow

```
sequenceDiagram
    actor User
    participant Browser
    participant AuthAPI as Auth API
    participant DB as Database
    User->>Browser: Register email/password
    Browser->>AuthAPI: POST /api/auth/register
    AuthAPI->>AuthAPI: Normalize email and hash password
    AuthAPI->>DB: Store user and hash
    User->>Browser: Submit login
    Browser->>AuthAPI: POST /api/auth/login
    AuthAPI->>DB: Find user and verify hash
    AuthAPI-->>Browser: Signed JWT + expiry
    Browser->>Browser: Store token in sessionStorage
    Browser->>AuthAPI: Protected request with Bearer token
    AuthAPI->>DB: Verify token subject and active user
    AuthAPI-->>Browser: Authorized response or 401
```

Passwords use `pwdlib.PasswordHash.recommended()`; current tests confirm Argon2id-format output. JWT subject is a user UUID, expiration is enforced, and `JWT_SECRET_KEY` is loaded as `SecretStr`. The default algorithm is HS256. Frontend `sessionStorage` is JavaScript-readable, so XSS can expose a live token. Logout removes the browser value only; there is no server-side revocation/refresh endpoint.

Ownership and File Authorization

```
flowchart TD
    Req[Request with bearer token and job UUID] --> User[Resolve active
user]
    User --> Query[Query by job UUID AND user UUID]
    Query -->|no row| NotFound[404 Download not found]
    Query -->|owned row| State{File request?}
```

```
    State -->|no| Detail[Return owner-scoped job/status]
    State -->|yes| Expiry{Completed and not expired?}
    Expiry -->|no| Reject[409 not ready or 410 expired]
    Expiry -->|yes| Path[Validate relative generated storage path]
    Path --> File[Return FileResponse]
```

History is filtered by `user_id`. Job detail, status, delete, and file routes call `get_download_for_user`. Foreign and nonexistent IDs share `404`, preventing direct object reference enumeration from disclosing ownership. Tests cover cross-user job read/delete and completed-file access.

SSRF Validation

```
flowchart TD
    URL[URL received] --> Scheme[Require HTTP or HTTPS]
    Scheme --> Host[Parse hostname, credentials, port]
    Host --> Local[Reject local names and unsupported ports]
    Local --> Resolve[Resolve hostname]
    Resolve --> Classify[Require every resolved IP to be global]
    Classify --> Guard[Enable connection-time getaddrinfo guard]
    Guard --> Request[Make provider request / yt-dlp extraction]
    Request --> Guard2[Guard every resolver result during operation]
    Guard2 --> Result[Return metadata or download]
```

`YtDlpProvider.validate_url()` resolves names and rejects non-global IPs; a `ContextVar`-scoped wrapper around `socket.getaddrinfo` repeats address classification during network activity. URL credentials are rejected; yt-dlp permits only default HTTP/HTTPS ports and rejects local suffixes and `/playlist`. The direct provider is additionally limited to exact allowlisted HTTPS hosts and `.mp4`, and disables redirects for both `HEAD` and `GET`.

Proxy caveat: yt-dlp receives `proxy=''`. Direct-file requests use `httpx.Client` without setting `trust_env=False`, so environment proxy variables may affect routing. The DNS guard cannot independently validate a destination resolved by an external proxy. This should be included in deployment SSRF/egress review.

Additional Controls

Risk

Control

Relevant implementation

Arbitrary selectors/options

Format/quality allowlist maps to server-owned selector plans; no user yt-dlp options

`../backend/app/providers/ytdlp_provider.py`

Playlist expansion

`noplaylist=True`; reject playlist/multi-video results

Same provider

Oversized source

Metadata filtering, provider size cap, worker progress/final checks

Providers and `../backend/app/workers/download_worker.py`

Path traversal

Relative path shape/UUID/extension validation and resolved containment

`../backend/app/services/storage_service.py`

Unsafe output file

UUID naming, `0700` directories, `0600` file, symlink/empty/residual checks

Storage service and worker

Invalid media

ffprobe container/stream validation with 15-second timeout

Worker

Upstream/DB detail leaks

Provider error mapping, generic job error, generic SQLAlchemy 503

`../backend/app/providers/errors.py`, `../backend/app/main.py`

Retained files

Expiration timestamp, lazy expiry, callable orphan cleanup

`../backend/app/services/download_service.py`, `storage_cleanup.py`

Security Boundaries and Gaps

HTTPS is mandatory for direct MP4; yt-dlp accepts HTTP or HTTPS public URLs.
No cookies, credentials, custom HTTP headers, or arbitrary yt-dlp options are provided. yt-dlp proxy is disabled. The app does not bypass DRM, login/access requirements, CAPTCHA, or rate limits.
Per-job acknowledgement is self-attestation, not authorization verification.
There is no rate limiting, CSRF-sensitive cookie flow (bearer token is used), production CSP/deployment config, monitoring, or scheduled cleanup defined here.
CORS includes configured frontend origin and development localhost patterns; narrow this for production.
Local temp storage and process-local queue impose deployment constraints.

Local Development Setup

Instructions are verified against the current scripts/configuration; commands target macOS. Use safe local credentials and never commit `.env` files.

Prerequisites

Node.js/npm compatible with the installed Vite 8 release.
Python with `venv` and `pip`; no minimum Python version is declared in the repository.
Docker Desktop or a local PostgreSQL 17-compatible server.
FFmpeg tools on `PATH`; `brew install ffmpeg` installs FFmpeg and ffprobe on macOS.

Configure Environment

From the repository root:

```
cp backend/.env.example backend/.env
cp .env.example .env
```

Edit `backend/.env` locally. Replace the JWT placeholder with a strong random secret. Set `POSTGRES_DB`, `POSTGRES_USER`, and `POSTGRES_PASSWORD`, then make `DATABASE_URL` use the same values. The Compose file explicitly requires all three `POSTGRES_*` variables. The root `.env` can retain the local frontend API URL or use `VITE_API_BASE_URL` to point to the backend.

Start Database and Backend

```
brew install ffmpeg
docker compose --env-file backend/.env up -d

cd backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
alembic upgrade head
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

The normal requirements omit yt-dlp. To opt into yt-dlp for sources you are authorized to download and the source permits:

```
cd backend
source .venv/bin/activate
pip install -r requirements-ytdlp.txt
```

Then set `YTDLP_ENABLED=true` in `backend/.env` and restart the API. With the example setting `false`, the provider is not selected.

Start Frontend

In another terminal from the repository root:

```
npm install
npm run dev
```

Open the Vite URL, normally `http://localhost:5173`. API defaults to `http://localhost:8000`; OpenAPI UI is `/docs`.

Checks and Shutdown

```
# Backend terminal, from backend/
pytest

# Frontend terminal, from repository root
npm run lint
npm run build

# Stop only the local Compose service
docker compose --env-file backend/.env down
```

Compose keeps PostgreSQL data in a named Docker volume. Do not use volume-removal options unless intentionally deleting local database contents. Do not run production with example placeholders, local CORS assumptions, or a development server.

Testing and Validation

Commands

Backend (run from `backend/` with the virtual environment active):

```
pytest
pytest tests/test_ytdlp_provider.py
```

Frontend (run from repository root):

```
npm run lint
npm run build
```

`npm run build` runs `tsc -b` followed by `vite build`. There is no frontend test script. To inspect migration graph without connecting to a database: `alembic heads` from `backend/`.

Current Audit Results

Check

Result

Backend pytest

92 passed in 1.65s; one Starlette deprecation warning that `httpx` with `starlette.testclient` is deprecated and `httpx2` is suggested

Frontend lint

`npm run lint` completed with no reported findings

Frontend build

`tsc -b` and Vite 8.3.2 build succeeded; 1,914 modules transformed

Migration head

`0003_download_progress (head)`

Live media requests

`https://samplelib.com/mp4/sample-5s.mp4` analyzed and completed as an authenticated `mp4/source` job; History showed the completed job and the authenticated file endpoint returned HTTP 200. Unauthenticated status/file requests returned HTTP 401. No Instagram or YouTube live tests were run.

The test run used the repository virtual environment (Python 3.14 was shown in the warning path). The repository does not declare a Python support range. Results describe this environment at audit time and should not be treated as compatibility certification.

Test Design and Scope

`backend/tests/conftest.py` creates schema in a static in-memory SQLite database per test, overrides API DB dependencies, disables actual background enqueueing, and disables yt-dlp by default. Provider tests supply fake HTTP responses; yt-dlp tests use `FakeYoutubeDL`; DNS answers are mocked. Worker tests use fake provider output, and ffprobe is mocked where output validation is under test.

Category

Evidence

Auth and password rules

`test_auth.py`

Owner filtering and IDOR

`test_auth.py`, `test_download_worker.py`

Download CRUD/history/status

`test_downloads.py`, `test_download_worker.py`

Health and safe DB errors

`test_health.py`, `test_downloads.py`

URL validation/provider registry

`test_providers.py`, `test_ytdlp_provider.py`

Direct provider MIME/redirect/network

`test_providers.py`

SSRF and connection-time DNS

`test_providers.py`, `test_ytdlp_provider.py`

yt-dlp options, qualities, playlist, errors

`test_ytdlp_provider.py`

Worker transitions/progress/size/failure

`test_download_worker.py`

ffprobe output validation

`test_download_worker.py`

Path traversal, permissions, expiration, cleanup

`test_download_worker.py`

Not Established by Tests

No test proves a real external extractor remains available, downloads content from a live source, or behaves consistently with changing platform rules. No frontend browser/e2e, accessibility, device, load/concurrency, production PostgreSQL migration, scheduler, or real FFmpeg/ffprobe integration test is configured. The suite verifies the application around controlled doubles, not external service availability.

Development History

Evidence Available

The supplied workspace has no `.git` directory, so commit history, dates/authors, and prior phase labels could not be inspected. Existing root/backend READMEs describe the current provider/configuration/security model but do not contain a detailed phase log. The test suite contains fixture URLs but uses fake providers, HTTP responses, fake yt-dlp, and mocked DNS; those fixtures are not live-test evidence.

The strongest chronological artifacts are the Alembic revisions:

Schema sequence

Verified change

`0001_initial`

User and download-history tables with status, owner foreign key, and initial metadata fields.

`0002_user_auth`

Password hash and active-account fields; email normalization and case-insensitive uniqueness.

`0003_download_progress`

Downloaded bytes, total bytes, and percentage columns.

Current code also verifies DirectFileProvider, a provider registry, optional generalized yt-dlp integration, JWT authentication, ownership checks, local temporary storage, progress reporting, ffprobe validation, a connection-time DNS guard, cleanup logic, and automated tests. Current presence does not establish which historical phase introduced each feature.

Phase 1–12 Reconstruction

Exact implementation history could not be fully reconstructed from repository evidence. No reliable source supports assigning the known implementation themes to numbered project phases. The three migration numbers describe schema revisions only and should not be mistaken for a complete project timeline. In particular, there is no verifiable historical record here of successful live YouTube/Instagram extraction, product releases, or production deployment.

Historical Live-Test Evidence

No run log, report, saved response, or automated live integration test was found. Test values such as a sample MP4 URL, an Instagram Reel path, and a YouTube-shaped URL appear in mocked tests. They establish validation behavior and test coverage, not a successful remote download. This audit itself made no live external media requests.

Phase 13 Report: UI/UX Polish and Real-User Validation

1. Objective

The Phase 13 task focused on frontend-only UX and accessibility improvements while preserving the backend API contract, database schema, authentication model, and provider/security controls. The goal was to make the app feel polished, clearer, and more resilient without changing the working backend architecture.

2. Scope guardrails

- No backend rewrites, schema changes, or provider logic rewrites.
- No new providers, auth bypasses, or security weakening.
- No removal of existing tests.
- UI changes had to remain truthful about what the app actually supports.
- Any runtime issue tied to an unavailable backend had to surface a clear connection error instead of a misleading user-facing message.

3. Frontend files updated

Major work landed in: - src/lib/api.ts - src/components/Navbar.tsx - src/components/Hero.tsx - src/components/Features.tsx - src/components/Footer.tsx - src/components/UrlDownloader.tsx - src/components/MediaResult.tsx - src/pages/Home.tsx - src/pages/History.tsx - src/pages/Login.tsx - src/pages/Register.tsx - src/index.css - README.md

4. Implemented UX improvements

- Removed dead placeholder navigation/footer links.
- Added real anchor navigation to `/#download` and `/#features`.
- Improved mobile navigation accessibility (`aria-expanded`, `aria-controls`, `aria-current`).
- Tightened hero and feature copy to avoid unsupported or exaggerated claims.
- Removed the redundant platform list that implied broader coverage than the app guarantees.
- Improved form error states and validation messaging for URL and auth flows.
- Increased touch-target sizes and focus-visible styling for keyboard access.
- Reworked the result card for clearer statuses, format details, progress, and completion affordances.

5. Implemented connection-error handling

A centralized API wrapper now catches network-level failures and presents a clear user

message: “Unable to connect to DownloadIt. Check the API connection and try again.” This replaced leaking raw browser `TypeError` messages and made the offline/backend-down experience consistent across the app.

6. Implemented URL validation hardening

The analyzer now explicitly requires `http://` or `https://` before a request is sent. Unsupported schemes fail immediately with a clear validation message rather than making a network request.

7. Implemented auth-page polish

Login and registration pages received stronger accessibility wiring, better callout styling for errors, clearer helper text, and consistent focus treatment. The password requirement guidance remains aligned with the backend password policy.

8. Implemented history page responsiveness

The previous mobile-hostile table layout was replaced by card-based list presentation that scales better on small screens and makes status/actions clearer without horizontal scrolling.

9. Implemented result-card state redesign

The analysis result component was redesigned for: - idle/ready state - in-progress polling - failed/ expired state - success/completed state - authorization acknowledgment callout - format details display using backend-provided values only

This removed a misleading mock-only state machine and better matched the app’s actual server contract.

10. Implemented accessibility updates

`prefers-reduced-motion` handling was added to reduce unnecessary visual movement.
Focus-visible outlines were applied to interactive actions.
Better `aria-*` labeling was introduced on the mobile menu and form fields/error messages.
Touch targets for buttons and selects were restored to a usable minimum height.

11. Documented behavior updates

The project documentation was updated to reflect the real frontend behavior: - the frontend surfaces a connection error if the API is unreachable - the UI shows only backend-provided format details and never fabricates unsupported format qualities - the Compose startup path now uses a dedicated Compose env file

12. Validation status: frontend lint

Executed: `npm run lint` Result: passed with no findings.

13. Validation status: frontend build

Executed: `npm run build` Result: passed (`tsc -b && vite build`), producing a successful production bundle.

14. Validation status: backend tests

Executed: `cd backend && .venv/bin/pytest -q` Result: 92 passed in 1.60s.

15. Live verification performed

The application was verified against the live backend with the stack running locally: - Docker/Postgres became healthy - Alembic reached head migration state - FastAPI served successfully on `http://127.0.0.1:8000` - Vite served successfully on `http://127.0.0.1:5173` - registration succeeded - sign-in succeeded - history view loaded correctly for an authenticated user - the allowlisted Direct MP4 sample `https://samplelib.com/mp4/sample-5s.mp4` analyzed successfully as `video/mp4` (2,848,208 bytes) - the sample was submitted as `mp4/source`; the job completed and the UI displayed “Download ready” - the completed job appeared in authenticated History with the expected source URL, direct platform, MP4 format, and source quality - clicking the authenticated file action returned HTTP 200 from the file endpoint - unauthenticated requests to that job’s status and file endpoints both returned HTTP 401 - offline/error state had previously displayed the new connection message rather than a misleading credential or fetch failure

16. Connection and infrastructure note

The environment had a real local infra wrinkle: `docker-compose.yml` expects `POSTGRES_DB`, `POSTGRES_USER`, and `POSTGRES_PASSWORD` via Compose env injection, while the app settings loader in `backend/app/core/config.py` rejects those as unknown extra fields when they are placed in `backend/.env`.

To keep the backend config clean and avoid backend source changes outside the requested scope, the compose-only values were moved to a separate backend/.env.docker file, which is then supplied via `docker compose --env-file backend/.env.docker up -d`.

17. What was not live-tested

The following were not tested: - Instagram Reel or YouTube analysis/download; no claim is made about their current availability or support - cross-user live access to the completed job; unauthenticated status/file access was tested, while owner isolation is covered by the backend tests - whether the integrated browser harness saved the returned file to a local download directory; its download-event observer did not report an event, although clicking the UI action produced an HTTP 200 file response

The Direct MP4 test covers only the specific public sample URL at the time of testing and does not establish general provider or platform support.

18. Risk and confidence assessment

Confidence is high for the frontend polish changes because the app was rebuilt successfully, the backend suite still passes, and the authenticated Direct MP4 analyze → download → History →

file-endpoint flow was verified live. Confidence is limited to the tested Direct MP4 sample; yt-dlp extractors and other platforms were not tested live.

19. Final outcome

Phase 13 is complete for the requested frontend polish and the live Direct MP4 flow. The app remains honest about supported media behavior, and the user-facing network/error states are clearer and more consistent.

20. Recommended next phase

If broader live-media verification is needed, run separate tests only against media the tester is authorized to access and the source permits, and report each exact URL and outcome without extrapolating support to other platforms.

Production Readiness Assessment

This is an assessment, not an implementation plan. The app is suitable for continued local development and isolated functional tests; production readiness is not established.

Area

Current evidence

State

Recommended work

Frontend

React app builds and lints; placeholder links; no browser tests

Partial

UX/accessibility review, real-user tests, browser tests, remove placeholder destinations.

Backend

FastAPI route/service separation and safe error handling

Partial

Deploy behind hardened ingress; define operations, timeouts, and observability.

Database

PostgreSQL Compose service and three migrations

Partial

Production DB provisioning, migration runbook, backups and restore tests.

Authentication

Password hashes and 30-minute default JWT

Partial

Review token storage, revocation/refresh needs, account abuse controls.

Authorization

Owner-scoped lookup for job/history/file routes

Implemented

Keep IDOR tests as routes expand.

Downloader

Allowlisted direct MP4; optional yt-dlp, fake-provider tests

Partial

Authorized live validation in a controlled environment; resource and concurrency isolation.

Storage

Local private temp dirs/files and containment checks

Partial

Durable/shared strategy if needed, capacity quotas, automated cleanup and monitoring.

SSRF

Public IP precheck and connection-time DNS guard

Partial

Evaluate egress and proxy behavior; direct httpx can honor environment proxies.

Secrets

`SecretStr`, `.env` ignored, examples use placeholders

Partial

Managed secret store/injection, rotation, deployment validation.

CORS

Configured origin plus explicit localhost patterns and regex

Not production-ready as-is

Restrict origins/methods/headers to deployed frontend.

Rate limiting

Not present

Not started

Add abuse controls for auth, analysis, and job creation.

Logging/monitoring

No production observability stack found

Not started

Structured logs, metrics, tracing, alerting and retention policy.

Queue

FastAPI in-process background tasks

Not production-ready for durable work

Durable broker/worker, retry/idempotency, cancellation and concurrency controls.

Deployment/HTTPS

Compose runs PostgreSQL only

Not started

Deploy API/UI, TLS, hardened network/egress policy, health and rollout strategy.

Backups	Not configured	Not started	Database and any retained-file backups with recovery exercises.
Cleanup	Callable service and lazy expiry checks; no scheduler	Partial	Schedule cleanup and alert on failures/storage growth.
Tests	92 backend tests, mostly mocks; lint/build pass	Partial	Live integration in controlled/authorized sources, frontend/e2e, PostgreSQL and concurrency tests.

Operational Risks

- Jobs are not durable across process termination and there is no cross-process queue coordination.
- Cleanup has no automatic invocation. Completed files are expired lazily when status/file endpoints are used; history does not itself expire rows.
- Local filesystem storage is not shared across API replicas and the per-file cap does not constitute a total storage quota.
- No rate limit is configured for registration, login, analysis, or job creation.
- `sessionStorage` is exposed to same-origin JavaScript and there is no refresh/revocation flow.
- CORS contains development localhost allowances; production origins should be explicitly narrowed.
- yt-dlp compatibility is external and changes over time. No platform-wide guarantee or live extraction SLA is evidenced.
- Direct provider requests do not opt out of environment proxies, while the DNS guard cannot independently validate proxy-side resolution.

Suggested Sequence

- Define production threat model, legal scope, source policy, and deployment boundary.

Address token/CORS/rate-limit/secret requirements and proxy-aware SSRF/egress policy.
Replace in-process jobs with durable execution and operational concurrency limits.
Schedule cleanup; define storage quotas, backups, retention, and recovery.
Add monitoring/logging and production deployment/TLS automation.
Add controlled live integration and frontend/end-to-end regression coverage.

These suggestions are not code changes made by the documentation task.

Documentation Audit

Repository Inspected

- Root configuration and documentation: `README.md`, `package.json`, `package-lock.json`, `vite.config.ts`, TypeScript configs, `index.html`, `.gitignore`, `.env.example`, `docker-compose.yml`.
- Frontend: `src/App.tsx`, entry point, all pages, auth context/storage, API client, types, URL downloader/result, navigation, supported-provider display, landing/features/footer, CSS and responsive styles.
- Backend: `backend/README.md`, both environment examples (not local secret values), requirements, Alembic config/environment/revisions, FastAPI startup, routes, dependencies, settings/database/network guard, models, schemas, providers, services, worker, and all test modules.
- Local `.env` values, temporary downloaded files, generated `dist`, `node_modules`, virtual-environment contents, and caches were not copied into documentation. The workspace has no `.git` metadata, so history commands were unavailable.

Documentation Created

`docs/PROJECT_DOCUMENTATION.md`
`docs/ARCHITECTURE.md`
`docs/API.md`
`docs/SECURITY.md`
`docs/DATABASE.md`
`docs/TESTING.md`
`docs/SETUP.md`
`docs/DEVELOPMENT_HISTORY.md`
`docs/PRODUCTION_READINESS.md`
`docs/DOCUMENTATION_AUDIT.md`

Current Validation Results

- Backend tests: **92 passed**, 1 warning, 1.65s. Warning: Starlette deprecates `httpx` use with `starlette.testclient` and suggests `httpx2`.
- Frontend lint: `npm run lint` completed with no reported findings.
- Frontend build: `npm run build` succeeded; TypeScript project build and Vite 8.3.2 production bundle completed.
- Database/migrations: `alembic heads` reports `0003_download_progress (head)`. No database migration was applied during the audit.

Live media checks: none run; no external media was downloaded.
Git history/status: unavailable because this workspace is not a Git repository.

Verified Implemented Features

React URL analysis UX, account pages, history list, local auth state, API client, quality selector, status polling and authenticated browser download.
FastAPI auth/media/download/history/status/file/delete/health routes; PostgreSQL-targeted SQLAlchemy models and three migrations.
Provider registry with direct MP4 provider and opt-in generic yt-dlp provider.
Worker progress persistence, temporary file creation, size limits, ffprobe validation and safe failure state.
Owner-filtered job operations and authenticated file retrieval.
Expiration timestamps, lazy status/file expiry, and callable cleanup for orphan/expired storage.

Verified Security Controls

Password hashing and JWT authentication, active-user lookup, owner-scoped DB queries, uniform 404 for unknown/foreign job IDs.
URL scheme/host/port checks, IP address classification, connection-time `getaddrinfo` guard, redirect denial for direct provider, yt-dlp proxy disablement.
Direct exact-host HTTPS MP4 allowlist; generated file/job IDs, path validation/containment, restrictive permissions, max-size checks.
yt-dlp playlists disabled, controlled format selectors, no cookies/credentials/custom headers/user-supplied options; output validation with ffprobe.
Public-safe provider/worker/database errors.

Verified Download Providers

`DirectFileProvider`: allowlisted HTTPS `.mp4` direct response with MIME/size validation; direct redirects disabled; only `mp4/source`.
`YtDlpProvider`: optional (`YTDLP_ENABLED=false` in example), general extractor attempt for permitted public HTTP(S) URLs; requires explicit per-job acknowledgement and offers controlled MP4/M4A outputs.
The source does not guarantee every yt-dlp extractor or website. Automated tests use fakes.

Verified Live-Test Evidence

Historical evidence found: **none**. URL-shaped test fixtures exist, but network clients and yt-dlp are mocked.
Tests executed during documentation task: backend pytest only against isolated test fixtures; frontend lint/build and Alembic head inspection. No live external media requests were made.

Incomplete / Planned Features

No production deployment, durable job queue, rate limiting, observability, automatic cleanup scheduler, or backup strategy was found.
No frontend unit/e2e suite was found.

UI links for Features/Platforms/Pricing/Privacy/Terms are placeholders.
No phase-by-phase development record or verified live extractor report is present.

Configuration Warnings

Never deploy with example JWT/database placeholders.
Compose requires explicit `POSTGRES_DB`, `POSTGRES_USER`, and `POSTGRES_PASSWORD`; these must agree with `DATABASE_URL`.
yt-dlp is a separate optional dependency and disabled by the example config.
`APP_ENV` is defined but no application behavior depending on it was found.
CORS includes localhost patterns intended for development.
Direct-file httpx may honor environment proxy variables; yt-dlp explicitly disables proxies.
Existing backend README calls media analysis authenticated, but the current analysis route is public. README no-proxy wording is accurate for yt-dlp but should not be generalized to DirectFileProvider's default httpx client.
The ignored local `.env` was intentionally not inspected; its actual yt-dlp setting and private values were not validated or reproduced.
Local environment files were not read or reproduced; no secret values are included in the docs.

Production-Readiness Warnings

Process-local jobs can be lost on restart and do not coordinate between replicas.
Cleanup is callable and expiry is lazy, but cleanup is not scheduled; local storage has no total quota or shared backend.
Session token in `sessionStorage` is JavaScript-readable, and no refresh/revocation flow exists.
No rate limiting, production logging/monitoring, deployment/TLS config, backup/restore plan, or production egress policy was found.
Test coverage is strong for mocked backend behavior but does not prove current external website compatibility.

Documentation Confidence

Area

Confidence

Basis / caveat

API routes and schemas

HIGH

All route modules and schemas inspected.

Database/migrations

HIGH

Models, all three migrations, Alembic config/head inspected.

Providers/quality/options

HIGH

Provider implementations and mocked tests inspected.

Authentication/ownership

HIGH

Routes, dependencies, services, frontend token handling and tests inspected.

Storage/worker/validation

HIGH

Implementation and focused worker tests inspected.

SSRF/proxy behavior

MEDIUM

Source and tests inspected; real network/proxy topology was not exercised.

Frontend behavior

HIGH

Entry, pages, components, API/auth/types/styles inspected; no browser run.

Live platform support

LOW

No live test evidence; external extractor behavior is intentionally not claimed.

Production operations/history

MEDIUM

Current files reviewed; no Git history or deployment artifacts available.